A PROJECT REPORT ON

AI-Powered Person Detection System

SUBMITTED BY

ABHIJEET GAIKWAD	EXAM NO. 22320144
PRADNYA TAGADE	EXAM NO. 22210038
SHANKAR ADGALE	EXAM NO. 22320023
JAYESH WANKHEDE	EXAM NO. 22320163

Under the Guidance of
Prof. Amar Buchade

In partial fulfilment of
Bachelor of Technology
Artificial Intelligence and Data Science
2025-2026
At

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

Affiliated To

Savitribai Phule Pune University, Pune

BRACT's
VISHWAKARMA INSTITUTE OF INFORMATION TECHNOLOGY, PUNE

CERTIFICATE

This is to certify that the project report entitled

"AI-Powered Person Detection System"

Submitted by

ABHIJEET GAIKWAD	EXAM NO. 22320144
PRADNYA TAGADE	EXAM NO. 22210038
SHANKAR ADGALE	EXAM NO. 22320023
JAYESH WANKHEDE	EXAM NO. 22320163

is a Bonafide student at this institute and the work has been carried out by them under the supervision of Prof. **Prema Kadam** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Technology** (AI & DS).

Prof. Amar Buchade
Guide

Prof. Santosh Kumar
Head of Department

Prof. Dr. Vivek Deshpande
Director, VIIT, Pune

Date:

Examiners : 1)…….. 2).....

Place: Pune

ACKNOWLEDGEMENT

We would like to express our sincere appreciation to all those who have contributed to the successful completion of this college project. This collaborative endeavor has been a journey of growth and learning, and we are grateful for the support and guidance we have received. First and foremost, we extend our heartfelt appreciation to our teachers for their invaluable guidance, mentorship, and unwavering support throughout the entire project. Their expertise and insights have been instrumental in shaping and refining our work.

We are thankful for the constructive feedback and scholarly input provided by all the professors. Their dedication to fostering academic excellence has been a source of inspiration for our group. Special thanks go to each member of our group for their dedication, hard work, and unique contributions. The collaborative spirit and diverse skill set within our team have been crucial to the success of this project.

We also acknowledge the assistance and resources provided by AI&DS Intelligence Dept, without which this project would not have been possible. The collaborative environment and access to facilities have greatly enhanced the overall quality of our work.

Lastly, we want to express our gratitude to our families for their unwavering support, patience, and understanding. Their encouragement has been a driving force behind our academic pursuits. Thank you to everyone who has played a role, big or small, in making this project a reality. Your contributions have been truly appreciated.

ABSTRACT

Real-time person detection and counting is a critical requirement in modern surveillance, crowd management, and smart building systems. Traditional methods relying on manual monitoring are error-prone, labor-intensive, and cannot scale effectively. This project presents an AI-powered person detection and counting system built using YOLOv12 for object detection, ByteTrack for multi-object tracking, and NVIDIA TensorRT for GPU-accelerated inference.

The system processes live video streams from RTSP cameras or recorded footage, detects persons in each frame, assigns persistent track identities across frames using motion-based association, and counts directional crossings over a configurable horizontal line to determine in/out movement.

The detection pipeline resizes frames to 640x640 pixels and applies YOLOv12 with a confidence threshold of 0.32, achieving real-time performance through TensorRT optimization. ByteTrack maintains consistent person identities using Kalman filter-based motion prediction and IoU-based association without requiring computationally expensive appearance features. The BoxMOT library (v12.0.8) provides a modular tracker zoo supporting eight tracking algorithms — including StrongSort, BotSort, and DeepOcSort — enabling comparative evaluation and flexible deployment.

A threaded video capture module ensures low-latency frame acquisition for live streams. The system demonstrates reliable person counting with real-time visualization of bounding boxes, track IDs, and cumulative in/out statistics, making it suitable for deployment in access control, retail analytics, and occupancy monitoring scenarios.

TABLE OF CONTENTS

LIST OF ABBREVIATIONS i
LIST OF FIGURES ii
LIST OF TABLES ii

Sr. No.		Title of Chapter		Page No.
01		**Introduction**		01
	1.1	Overview		01
	1.2	Motivation		05
	1.3	Problem Definition and Objectives		07
	1.4	Project Scope & Limitations		09
	1.5	Methodologies of Problem solving		11
02		**Literature Survey**		14
03		**Software Requirements Specification**		19
	3.1	Assumptions and Dependencies		19
	3.2	Functional Requirements		23
		3.2.1	System Feature 1(Functional Requirement)	23
		3.2.2	System Feature2 (Functional Requirement)	23
	3.3	External Interface Requirements (If Any)		25
		3.3.1	User Interfaces	25
		3.3.2	Hardware Interfaces	26
		3.3.3	Software Interfaces	26
	3.4	Nonfunctional Requirements		27
		3.4.1	Performance Requirements	27
		3.4.2	Security Requirements	27
		3.4.3	Software Quality Attributes	28
	3.5	Analysis Models: SDLC Model to be applied		29
04		**System Design**		31
	4.1	System Architecture		31
	4.2	UML Diagrams		32
05		**Project Plan**		38
	5.1	Project Estimate		38
		5.1.1	Reconciled Estimates	38
		5.1.2	Project Resources	39
	5.2	Risk Management		40
		5.2.1	Risk Identification	40
		5.2.2	Risk Analysis	40

Sr. No.		Title of Chapter		Page No.
		5.2.3	Overview of Risk Mitigation, Monitoring, Management	41
	5.3	Project Schedule		41
		5.3.1	Project Task Set	41
		5.3.2	Task Network	42
		5.3.3	Timeline Chart	44
	5.4	Team organization		45
		5.4.1	Team structure	45
		5.4.2	Management reporting and communication	45
06			**Project Implementation**	46
		6.1	Overview of Project Modules	46
		6.2	Tools and Technologies Used	47
		6.3	Algorithms Details	48
07		**Software Testing**		49
		7.1	Type of Testing	49
		7.2	Test cases & Test Results	50
08		**Results**		52
		8.1	Outcomes	52
		8.2	Screen Shots	53
09		**Conclusions**		58
		9.1	Conclusions	58
		9.2	Future Work	60
		9.3	Applications	62
10		**References**		63

LIST OF FIGURES

FIGURE	ILLUSTRATION	PAGE NO.
4.2.1	System Architecture	32
4.2.2	System Overview	33
4.3.1	Use case Diagram	34
4.3.2	Class Diagram	35
4.3.3	Sequence Diagram	36
4.3.4	Activity Diagram	37

LIST OF TABLES

TABLE	ILLUSTRATION	PAGE NO.
5.3.3	Time- Line chart	44

CHAPTER 1 : INTRODUCTION

1.1 OVERVIEW -

In today's rapidly evolving technological landscape, the demand for intelligent surveillance and monitoring systems has grown significantly across a wide range of sectors including public safety, transportation, retail analytics, smart buildings, and industrial environments. The ability to detect, track, and count people in real time from video feeds has become a foundational requirement for applications such as crowd management, access control, occupancy monitoring, and security enforcement. Traditional approaches to these tasks have relied heavily on manual human observation, which is inherently limited by fatigue, attentional lapses, subjectivity, and the inability to scale across multiple camera feeds simultaneously. These limitations have created a compelling need for automated, intelligent, and scalable systems capable of performing person detection and counting with high accuracy and minimal latency.

Object detection, one of the core problems in computer vision, involves identifying and localizing objects of interest within an image or video frame. Over the past decade, deep learning has revolutionized the field of object detection, with Convolutional Neural Networks (CNNs) serving as the foundational architecture for extracting spatial features from visual data. Early deep learning-based detectors such as R-CNN, Fast R-CNN, and Faster R-CNN introduced the concept of region-based detection, achieving high accuracy but at the cost of computational speed. These two-stage detectors, while effective, proved impractical for real-time applications due to their sequential region proposal and classification pipeline. The need for speed without significant accuracy loss led to the development of single-stage detectors, which perform detection in a single forward pass through the network.

Among single-stage detectors, the YOLO (You Only Look Once) family of models has emerged as the most widely adopted architecture for real-time object detection. YOLO treats detection as a regression problem, predicting bounding box coordinates and class probabilities directly from the full image in a single evaluation. Since the introduction of the original YOLO in 2016, the architecture has undergone significant evolution through successive versions — YOLOv2, YOLOv3, YOLOv4, YOLOv5, YOLOv7, YOLOv8, and most recently YOLOv12 — each bringing improvements in backbone design, feature pyramid networks, anchor-free detection heads, and training strategies. YOLOv12, the version employed in this project, represents the latest advancement in the YOLO

lineage, incorporating attention mechanisms and optimized architectural components that deliver superior detection accuracy while maintaining real-time inference speeds. Combined with the Ultralytics framework, YOLOv12 provides a streamlined interface for training, exporting, and deploying detection models across various hardware platforms.

However, detecting persons in individual frames is only the first step toward meaningful surveillance intelligence. In real-world video streams, the same person appears across multiple consecutive frames, and the system must maintain a consistent identity for each individual over time. This problem is known as Multi-Object Tracking (MOT), and it introduces challenges such as occlusion (when persons partially or fully block each other), identity switching (when the tracker incorrectly swaps IDs between individuals), re-identification after temporary disappearance, and handling varying speeds and directions of movement. Multi-object tracking algorithms bridge the gap between per-frame detection and temporal understanding by associating detections across frames and assigning persistent identifiers to each tracked individual.

The tracking component of this project is built upon ByteTrack, a state-of-the-art multi-object tracking algorithm that distinguishes itself by utilizing both high-confidence and low-confidence detections for track association. Unlike conventional trackers that discard low-confidence detections — potentially losing track of partially occluded or distant persons — ByteTrack performs a two-stage association process. In the first stage, high-confidence detections are matched to existing tracks using IoU (Intersection over Union) similarity. In the second stage, unmatched tracks are associated with remaining low-confidence detections, significantly reducing the rate of missed tracks and identity switches. ByteTrack relies entirely on motion-based association using a Kalman filter for state prediction, making it computationally lightweight and free from the overhead of appearance-based Re-Identification (ReID) models. This design choice makes ByteTrack particularly well-suited for real-time applications where inference speed is critical.

To achieve real-time performance on high-resolution video streams, this project leverages NVIDIA TensorRT, a high-performance deep learning inference optimizer and runtime library. TensorRT takes the trained YOLOv12 model and optimizes it through layer fusion, precision calibration (FP16 half-precision), kernel auto-tuning, and memory optimization to maximize throughput on NVIDIA GPUs. The YOLOv12 model is first exported from its native PyTorch format to a TensorRT engine file,

which is then loaded at runtime for inference. This optimization pipeline significantly reduces inference latency compared to running the model directly in PyTorch, enabling the system to process video frames at rates suitable for real-time surveillance applications.

Beyond detection and tracking, the system implements a directional person counting mechanism based on crossing-line detection. A horizontal reference line is defined at a configurable position within the frame (set at 55% of the frame height in the current implementation). The system monitors the bottom-center point of each tracked person's bounding box and maintains a state machine for each track ID, classifying the person as either "inside" (below the line) or "outside" (above the line). When the system detects a state transition — a person's position moving from outside to inside or vice versa — it increments the corresponding IN or OUT counter. This approach provides a simple yet effective mechanism for monitoring bidirectional foot traffic through doorways, corridors, gates, or any defined boundary within the camera's field of view.

The project further incorporates the BoxMOT library (version 12.0.8), a comprehensive and modular multi-object tracking framework that provides a zoo of eight different tracking algorithms: ByteTrack, BotSort, StrongSort, OcSort, DeepOcSort, HybridSort, ImprAssoc, and BoostTrack. Each tracker offers a different balance between speed and accuracy, with some relying purely on motion cues (ByteTrack, OcSort) and others incorporating appearance-based Re-Identification features for more robust identity maintenance (StrongSort, BotSort, DeepOcSort, HybridSort, ImprAssoc, BoostTrack). The BoxMOT library also includes supporting subsystems for Kalman filter-based motion prediction, camera motion compensation (using techniques such as ECC, ORB, SIFT, and sparse optical flow), and ReID model inference across multiple backends including PyTorch, ONNX, TensorRT, OpenVINO, and TFLite. This modular architecture enables comparative evaluation of different tracking strategies and provides flexibility for deployment across diverse hardware configurations and application requirements.

The visualization component of the system uses OpenCV to render real-time annotations on the video feed, including bounding boxes around detected persons, unique track ID labels, confidence scores, the crossing reference line, and cumulative IN/OUT count statistics. This visual feedback enables operators to monitor the system's performance in real time and verify the accuracy of detection, tracking, and counting operations. A threaded video capture module is also available within the

codebase, providing non-blocking frame acquisition from RTSP camera streams through a daemon thread, which ensures low-latency frame delivery for latency-sensitive deployments.

In summary, this project presents a complete end-to-end pipeline for real-time person detection, multi-object tracking, and directional counting. By combining the detection power of YOLOv12, the tracking efficiency of ByteTrack, the inference optimization of TensorRT, and the modular flexibility of the BoxMOT tracking framework, the system delivers a practical and deployable solution for intelligent surveillance and occupancy monitoring. The following sections of this chapter elaborate on the motivation behind this work, the specific problem definition and objectives, the scope and limitations of the project, and the methodologies employed for problem solving.

1.2 MOTIVATION -

The motivation behind this project stems from the growing need for intelligent, automated monitoring systems that can replace or augment manual human surveillance in a variety of real-world environments. In today's world, video surveillance cameras are ubiquitous — installed in shopping malls, airports, railway stations, corporate offices, hospitals, educational institutions, and public spaces. However, the vast majority of these cameras serve merely as passive recording devices, with their footage reviewed only after an incident has occurred. The potential for real-time, proactive intelligence from these cameras remains largely untapped. This project is motivated by the vision of transforming passive surveillance infrastructure into an active, intelligent system capable of detecting, tracking, and counting people in real time.

One of the primary motivations is the challenge of crowd management and occupancy monitoring. In the wake of global events such as the COVID-19 pandemic, there has been a heightened awareness of the need to monitor and control the number of people in enclosed spaces. Building managers, retail store operators, and public venue administrators require accurate, real-time occupancy data to enforce capacity limits, optimize space utilization, and ensure public safety. Manual counting methods — such as turnstiles, clicker counters, or security personnel stationed at entry points — are labor-intensive, error-prone, and cannot scale effectively. An automated vision-based counting system addresses these limitations by providing continuous, accurate, and non-intrusive occupancy monitoring without requiring any physical interaction with the individuals being counted.

Another key motivation is the advancement of real-time inference capabilities through GPU acceleration. The availability of NVIDIA TensorRT and modern GPU hardware has made it feasible to deploy complex deep learning models in real-time video processing pipelines. Models like YOLOv12, which would have been impractical for real-time use on earlier hardware, can now process video frames at high frame rates when optimized with TensorRT. This convergence of powerful detection models and efficient inference engines creates an opportunity to build practical, deployable systems that were previously confined to research environments.

The rapid evolution of multi-object tracking algorithms also serves as a significant motivation.

Traditional tracking approaches often suffered from high rates of identity switching, poor handling of occlusions, and excessive computational overhead. The emergence of algorithms like ByteTrack — which achieves state-of-the-art tracking performance using only motion cues without requiring computationally expensive appearance models — has made robust multi-object tracking accessible for real-time applications. The availability of the BoxMOT library, which packages eight different state-of-the-art trackers into a unified framework with configurable hyperparameters, further motivates the exploration and comparison of different tracking strategies for person counting applications.

Finally, this project is motivated by the academic objective of integrating multiple cutting-edge technologies — deep learning-based detection, motion-based tracking, GPU-accelerated inference, and computer vision-based counting — into a cohesive, functional system. The process of building such a system provides valuable learning opportunities in areas such as model optimization, real-time systems design, video processing pipelines, and the practical challenges of deploying AI in real-world environments.

1.3 PROBLEM DEFINITION AND OBJECTIVES -

Problem Definition:

Manual surveillance and person counting in public and private spaces is fundamentally limited by human attention capacity, fatigue, and the inability to simultaneously monitor multiple video feeds. Existing automated solutions often rely on specialized hardware (such as infrared sensors or turnstiles) that is expensive to install and maintain, or on software systems that fail to provide the real-time performance required for dynamic environments. There is a need for a cost-effective, software-based solution that can leverage existing camera infrastructure to perform real-time person detection, tracking, and bidirectional counting with high accuracy and minimal latency.

The specific technical challenges that this project addresses include:

1. Real-time person detection: Identifying all persons in each video frame with sufficient accuracy while maintaining real-time processing speeds. This requires balancing detection model complexity with inference speed, necessitating GPU-accelerated inference through TensorRT optimization.

2. Consistent multi-object tracking: Maintaining unique, persistent identity assignments for each detected person across consecutive video frames, despite challenges such as occlusion, overlapping trajectories, varying movement speeds, and persons entering or leaving the frame.

3. Accurate directional counting: Reliably detecting when a tracked person crosses a defined boundary line and correctly classifying the crossing direction (IN or OUT), while handling edge cases such as persons lingering near the line, partial crossings, and rapid movements.

4. System robustness: Ensuring stable operation across varying video conditions including different lighting environments, camera angles, frame rates, and video sources (RTSP streams, recorded video files, and webcam feeds).

Objectives:

The primary objectives of this project are as follows:

1. To develop a real-time person detection system using YOLOv12 with TensorRT optimization that can process video frames at rates suitable for live surveillance applications.

2. To implement multi-object tracking using the ByteTrack algorithm for consistent person identity assignment across video frames, leveraging Kalman filter-based motion prediction and IoU-based association.

3. To design and implement a crossing-line-based directional counting mechanism that accurately tracks IN and OUT movements of persons across a defined reference boundary.

4. To integrate the detection, tracking, and counting components into a unified, end-to-end video processing pipeline with real-time visualization of results.

5. To explore the BoxMOT tracking framework and its suite of eight tracking algorithms as configurable alternatives to ByteTrack for comparative evaluation and application-specific deployment.

6. To provide a threaded video capture mechanism for low-latency frame acquisition from RTSP camera streams, supporting deployment in live surveillance environments.

1.4 PROJECT SCOPE & LIMITATIONS

Scope:

The scope of this project encompasses the design, development, and demonstration of a real-time person detection and counting system with the following capabilities:

- Detection of persons in video frames using the YOLOv12 deep learning model, optimized for GPU inference through NVIDIA TensorRT with FP16 half-precision.
- Multi-object tracking using ByteTrack for persistent identity assignment, with the BoxMOT library providing access to seven additional tracking algorithms for comparative analysis.
- Directional person counting based on a horizontal crossing line, with real-time IN/OUT statistics displayed on the video feed.
- Support for multiple video input sources including RTSP camera streams, local video files, and USB webcam feeds.
- Real-time visualization with bounding boxes, track IDs, confidence scores, and counting statistics rendered using OpenCV.
- A threaded video capture module for non-blocking frame acquisition from network camera streams.
- Configurable parameters including detection confidence threshold, crossing line position, frame resolution, and tracker hyperparameters via YAML configuration files.

Limitations:

Despite its capabilities, the current system has the following limitations:

1. Single-camera operation: The system currently processes a single video stream at a time. Multi-camera support with cross-camera person re-identification is not implemented.

2. Fixed crossing line: The counting boundary is a horizontal line at a fixed vertical position (55% of frame height). Diagonal, curved, or dynamically positioned boundaries are not

supported.

3. GPU dependency: The TensorRT-optimized inference pipeline requires an NVIDIA GPU with CUDA support. The system cannot achieve real-time performance on CPU-only hardware.

4. No persistent storage: Detection and counting results are displayed in real time but are not logged to a database or file for historical analysis.

5. Single class detection: The system is configured to detect only persons (COCO class 0). Detection and tracking of other object classes (vehicles, animals, etc.) would require reconfiguration but not architectural changes.

6. Environmental sensitivity: Detection accuracy may degrade under extreme lighting conditions, heavy occlusion, unusual camera angles, or very high crowd density.

7. No Re-Identification: The default ByteTrack configuration uses motion-only association. Persons who leave and re-enter the frame are assigned new track IDs rather than being re-identified.

1.5 METHODOLOGIES OF PROBLEM SOLVING -

The development of this project follows an iterative, modular methodology that breaks the overall problem into discrete, manageable components — each addressing a specific aspect of the detection, tracking, and counting pipeline. The methodology is structured into the following phases:

Phase 1: Environment Setup and Model Preparation

The first phase involves setting up the development environment with the necessary software dependencies including Python, PyTorch, Ultralytics YOLO, OpenCV, NumPy, and the BoxMOT library. The YOLOv12 model, pre-trained on the COCO dataset (which includes the "person" class among 80 object categories), is exported from its native PyTorch format (.pt) to a TensorRT engine (.engine) using the Ultralytics export command with half-precision (FP16) enabled and an input image size of 640x640 pixels. This one-time export process optimizes the model for GPU inference on the target hardware.

Phase 2: Video Capture and Preprocessing

The second phase addresses the acquisition and preprocessing of video frames. The system uses OpenCV's VideoCapture interface to read frames from the configured video source (RTSP URL, local file path, or webcam index). Each captured frame is resized to a fixed resolution of 640x640 pixels to match the model's expected input dimensions and ensure consistent processing regardless of the original video resolution. The frame is converted from BGR to RGB color space as required by the YOLO model. A buffer size of 1 is set on the video capture to minimize latency, ensuring the system always processes the most recent frame.

Phase 3: Object Detection with YOLOv12

The third phase performs person detection using the TensorRT-optimized YOLOv12 model. The preprocessed frame is passed to the model's track() method, which internally performs

detection, non-maximum suppression, and tracking in a single call. The detection is filtered to the person class (COCO class index 0) and returns bounding box coordinates, confidence scores, and track IDs for each detected person. A confidence threshold of 0.32 is applied to filter out low-confidence detections that may represent false positives.

Phase 4: Multi-Object Tracking with ByteTrack

The fourth phase integrates ByteTrack for multi-object tracking, invoked through the Ultralytics model.track() interface with the bytetrack.yaml configuration file. ByteTrack maintains a Kalman filter state for each active track, predicting the expected position of each person in the current frame based on their motion history. Detections are associated with existing tracks using IoU similarity in a two-stage matching process — first matching high-confidence detections, then associating remaining low-confidence detections with unmatched tracks. New track IDs are created for unmatched detections, and tracks that have not been associated with any detection for a specified number of frames (max_age) are terminated.

Phase 5: Crossing Line Detection and Counting

The fifth phase implements the directional counting logic. A horizontal crossing line is defined at 55% of the resized frame height. For each tracked person, the system monitors the Y-coordinate of the bottom-center point of their bounding box (the y2 coordinate). Each track maintains a state machine with two states: "inside" (below the line) and "outside" (above the line). When the system detects that a person's position has transitioned from outside to inside (crossing downward past the line), the IN counter is incremented. Conversely, when a person transitions from inside to outside (crossing upward past the line), the OUT counter is incremented. The current occupancy is calculated as the difference between total IN and total OUT counts. Stale tracks (persons no longer visible) are cleaned up each frame to prevent memory leaks and state corruption.

Phase 6: Visualization and Output

The final phase renders the results on the video frame using OpenCV drawing functions.

Bounding boxes are drawn around each detected person in green, with the track ID and confidence score displayed as text labels. A circle marks the bottom-center tracking point of each person. The crossing reference line is drawn in red across the full width of the frame. Cumulative counts for Total In, Total Out, and Currently Inside are displayed in the upper-left corner. The annotated frame is displayed in a resizable window, and the processing loop continues until the user presses the 'q' key to exit.

This modular methodology allows each component to be developed, tested, and refined independently while maintaining a clear integration path for the complete pipeline. The iterative nature of the development process enables continuous improvement through parameter tuning (confidence thresholds, crossing line position, tracker hyperparameters) and component substitution (swapping ByteTrack for other trackers from the BoxMOT library) without requiring changes to the overall system architecture.

CHAPTER 2 : LITERATURE SURVEY

This chapter reviews the key technologies used in this project — YOLO object detection, multi-object tracking algorithms, TensorRT inference optimization, and vision-based person counting.

2.1 Object Detection: Classical to Deep Learning

Classical object detection relied on handcrafted features such as HOG (Dalal and Triggs, 2005) and Haar cascades (Viola and Jones, 2001) combined with SVM classifiers. While effective for constrained scenarios, these methods lacked robustness to variations in scale, pose, and occlusion.

Deep learning transformed object detection through Convolutional Neural Networks (CNNs). AlexNet (Krizhevsky et al., 2012) demonstrated the power of deep CNNs on ImageNet, followed by VGGNet, GoogLeNet, and ResNet which introduced deeper architectures with residual connections. Object detectors split into two families: two-stage detectors (R-CNN, Faster R-CNN) offering high accuracy but slow speed, and single-stage detectors that perform detection in a single forward pass for real-time performance.

2.2 The YOLO Family

YOLO (You Only Look Once), introduced by Redmon et al. (2016), redefined object detection as a single regression problem. Key milestones in its evolution:

- YOLOv1 (2016): Grid-based detection, 63.4% mAP at 45 FPS
- YOLOv2 (2017): Added batch normalization, anchor boxes, multi-scale training
- YOLOv3 (2018): Darknet-53 backbone with residual connections, multi-scale prediction via FPN
- YOLOv4 (2020): CSPDarknet53, PANet, mosaic augmentation
- YOLOv5 (2020): PyTorch-native by Ultralytics, multiple model sizes, easy export to

TensorRT/ONNX
- YOLOv7 (2022): E-ELAN for improved feature aggregation
- YOLOv8 (2023): Anchor-free detection head, unified detection/segmentation/pose framework

YOLOv12 (Tian et al., 2025), used in this project, is a paradigm shift — it incorporates attention mechanisms while maintaining real-time speed. Key innovations include:

- Area Attention (A^2): Partitions feature maps into segments, reducing self-attention complexity from $O(n^2)$ to $O(n)$
- R-ELAN: Residual Efficient Layer Aggregation Networks for stable training of deeper attention stacks
- FlashAttention: Minimizes memory access overhead
- 7×7 separable convolutions: Preserves spatial context with fewer parameters

YOLOv12-X achieves 55.2% mAP, surpassing YOLOv10-X by 0.8% at comparable speed.

2.3 Multi-Object Tracking Algorithms

Multi-Object Tracking (MOT) maintains consistent identities for detected objects across video frames. The dominant paradigm is tracking-by-detection. Key algorithms:

SORT (Bewley et al., 2016) combined a Kalman filter with the Hungarian algorithm for IoU-based track association, achieving 200+ FPS. However, it suffered frequent identity switches during occlusions due to lack of appearance information.

DeepSORT (Wojke et al., 2017) extended SORT with a deep ReID model for appearance-based matching, significantly reducing identity switches but adding computational overhead.

StrongSORT (Du et al., 2023) upgraded DeepSORT with stronger detection (YOLOX), better ReID (OSNet), EMA-Kalman filter, and Gaussian-Smoothed Interpolation for missing detections.

OC-SORT (Cao et al., 2023) improved SORT by prioritizing actual observations over predictions, introducing Observation-Centric Re-Update to correct Kalman states after occlusion.

BotSort (Zhang et al., 2022) combined motion and appearance with camera motion compensation for robust tracking in moving-camera scenarios.

DeepOCSORT (Liu et al., 2023) added Dynamic Appearance and Adaptive Weighting modules for tracking fast-moving, tiny objects.

2.4 ByteTrack

ByteTrack (Zhang et al., ECCV 2022) is the primary tracker in this project. Its key insight: conventional trackers discard low-confidence detections, losing track of occluded persons. ByteTrack associates every detection through a two-stage process:

1. First stage: Match high-confidence detections to tracks via IoU
2. Second stage: Match remaining low-confidence detections to unmatched tracks

ByteTrack uses only motion-based association (Kalman filter) — no ReID model needed — making it lightweight and fast. It achieved 80.3 MOTA, 77.3 IDF1, and 63.1 HOTA on MOT17 at 30 FPS on a V100 GPU. This combination of simplicity, speed, and accuracy makes it ideal for real-time person counting.

2.5 BoxMOT Framework

BoxMOT (Broström, 2024) is a modular multi-object tracking library providing eight trackers under a unified interface: ByteTrack, BotSort, StrongSort, OcSort, DeepOcSort, HybridSort, ImprAssoc, and BoostTrack. It categorizes trackers into motion-only (ByteTrack, OcSort) and motion-plus-appearance (the rest requiring ReID). The library includes Kalman filter variants, camera motion compensation (ECC, ORB, SIFT, sparse optical flow), ReID inference across multiple backends (PyTorch, ONNX, TensorRT,

OpenVINO, TFLite), and YAML-based hyperparameter configuration.

2.6 TensorRT Inference Optimization

NVIDIA TensorRT optimizes deep learning models for GPU inference through layer fusion, FP16/INT8 precision calibration, kernel auto-tuning, and memory optimization. In this project, the YOLOv12 model is exported from PyTorch (.pt) to a TensorRT engine (.engine) with FP16 half-precision. FP16 halves memory usage and doubles throughput on Tensor Core-equipped GPUs while TensorRT's mixed-precision strategy maintains accuracy by keeping sensitive layers in FP32.

2.7 Vision-Based Person Counting

Person counting approaches include:

- Detection-based counting (used in this project): Detect individuals, track them, count crossings over a boundary line. Provides trajectory and identity information.
- Regression-based counting (Chan et al., 2008): Maps image features directly to a count — effective for dense crowds but provides no individual tracking.
- Density estimation (Zhang et al., 2016 — CSRNet): Predicts density maps where integration gives count — effective at scale but no directional information.

The line-crossing paradigm adopted in this project defines a horizontal reference line in the frame. The system monitors each tracked person's position relative to the line and counts state transitions (outside→inside = IN, inside→outside = OUT). This provides bidirectional counting suitable for occupancy monitoring, retail analytics, and access control.

2.8 Summary

The technologies reviewed — YOLOv12 for detection, ByteTrack for tracking, TensorRT for inference optimization, and BoxMOT for modular tracker access — form the foundation of this project. The evolution from classical HOG-based detection to attention-enhanced YOLO, and from simple SORT to ByteTrack's every-box association, demonstrates the field's progress toward accurate, real-time person detection and counting systems.

CHAPTER 3 : SOFTWARE REQUIREMENTS SPECIFICATION

3.1 ASSUMPTIONS AND DEPENDENCIES

Assumptions:

1. Operating System
   - The project is developed and executed on a Windows 10+ or Linux environment with NVIDIA GPU drivers installed.
   - Linux provides better CUDA/TensorRT compatibility; Windows ensures accessibility for general users.

2. Development Environment
   - Python-based development using VS Code, PyCharm, or terminal-based execution.
   - No cloud-based notebook (e.g., Colab) is required — all inference runs locally on GPU hardware.

3. Programming Language
   - The entire project is written in Python 3.8+.
   - Python provides seamless integration with PyTorch, OpenCV, Ultralytics, and the BoxMOT library.

4. Deep Learning Framework
   - PyTorch is used as the primary deep learning framework for model loading and inference.
   - Ultralytics YOLO provides the high-level API for detection and tracking.

5. GPU and Inference Engine
   - An NVIDIA GPU with CUDA support is required for TensorRT-accelerated inference.
   - The YOLOv12 model is pre-exported to a TensorRT engine (.engine) with FP16 half-precision for optimized inference.

6. Video Input

- The system accepts video input from RTSP camera streams, local video files (MP4, AVI), or USB webcams.
- A stable network connection is required for RTSP stream input.

7. Model Availability
- The YOLOv12 pre-trained model (yolo12x.pt) trained on the COCO dataset is available and can be exported to TensorRT format using the Ultralytics CLI.

8. Real-Time Display
- A display monitor is available for OpenCV-based real-time visualization of detection, tracking, and counting results.

Dependencies:

The following third-party software and libraries are required:

- PyTorch (torch) — Deep learning framework for model inference and GPU computation.
- Ultralytics (ultralytics) — Provides the YOLO model API for detection, tracking, and model export.
- OpenCV (cv2) — Video capture, frame processing, resizing, color conversion, and visualization.
- NumPy (numpy) — Array operations for bounding box coordinate manipulation and numerical processing.
- NVIDIA TensorRT — Inference optimization engine for GPU-accelerated model execution.
- CUDA Toolkit — GPU programming interface required by PyTorch and TensorRT.
- BoxMOT (boxmot) — Multi-object tracking library providing ByteTrack, BotSort, StrongSort, and five other trackers.
- Loguru — Logging library for runtime event tracking and debugging.
- PyYAML (pyyaml) — YAML parser for reading tracker configuration files (bytetrack.yaml, etc.).
-

3.2 FUNCTIONAL REQUIREMENTS -

3.2.1 System Feature 1: Real-Time Person Detection and Tracking

The system must detect and track persons in real-time video streams using the YOLOv12 model and ByteTrack algorithm.

- The system accepts video input from RTSP URLs, local video files, or webcam (device index 0).
- Each frame is resized to 640×640 pixels and converted from BGR to RGB for model input.
- The YOLOv12 TensorRT engine performs person detection (COCO class 0) with a confidence threshold of 0.32.
- ByteTrack assigns persistent track IDs to each detected person across consecutive frames using Kalman filter prediction and IoU-based association.
- Bounding boxes, track IDs, and confidence scores are rendered on each frame in real time.
- Stale tracks (persons no longer visible) are automatically removed each frame.

3.2.2 System Feature 2: Directional Person Counting

The system must count persons crossing a horizontal reference line and classify each crossing as IN or OUT.

- A horizontal crossing line is defined at 55% of the resized frame height (Y = 352 for 640-height frames).
- The system monitors the bottom-center point (cx, y2) of each tracked person's bounding box.
- Each track maintains a state machine with two states: "inside" (below line) and "outside" (above line).
- When a person transitions from outside to inside (y-coordinate crosses downward past the line), the IN counter increments.
- When a person transitions from inside to outside (y-coordinate crosses upward past the line), the OUT counter increments.
- Current occupancy is calculated as: Currently Inside = Total In - Total Out.
- All counts are displayed on the video feed in real time.

3.3 EXTERNAL INTERFACE REQUIREMENTS -

3.3.1 User Interfaces

- The system provides a real-time OpenCV display window ("Local Processing - TensorRT") showing the annotated video feed.
- Bounding boxes (green) are drawn around detected persons with track ID and confidence labels.
- A red horizontal line indicates the crossing boundary.
- Count statistics (Total In, Total Out, Currently Inside) are displayed in the upper-left corner.
- The user presses the 'q' key to exit the application.
- The window is resizable using cv2.WINDOW_NORMAL.

3.3.2 Hardware Interfaces

- NVIDIA GPU: Required for TensorRT inference. Must support CUDA and have sufficient VRAM for the YOLOv12-X model.
- IP Camera: The system interfaces with network cameras via RTSP protocol (e.g., rtsp://admin:password@ip:port/stream).
- USB Webcam: Supported via OpenCV's VideoCapture with device index 0.
- Display Monitor: Required for real-time visualization output.

3.3.3 Software Interfaces

- Ultralytics YOLO API: The system calls model.track() for combined detection and tracking in a single inference pass.
- ByteTrack Configuration: Tracker behavior is configured via bytetrack.yaml with parameters including track_thresh (0.6), match_thresh (0.9), track_buffer (30), and frame_rate (30).
- OpenCV VideoCapture: Interfaces with video sources using cv2.CAP_FFMPEG backend for RTSP streams.
- BoxMOT Tracker Zoo

3.4 NONFUNCTIONAL REQUIREMENTS –

3.4.1 Performance Requirements

- The system must process video frames at near real-time speeds (target: 15-30 FPS depending on GPU capability).
- Detection inference latency should not exceed 50ms per frame with TensorRT FP16 optimization.
- Video capture buffer size is set to 1 to minimize latency and ensure the system always processes the most recent frame.
- The system should handle continuous operation for extended durations without memory leaks, achieved through stale track cleanup each frame.

3.4.2 Security Requirements

- RTSP stream credentials (username/password) are configured directly in the source code. In production deployment, these should be externalized to environment variables or a secure configuration file.
- The system processes video locally — no frames or detection data are transmitted to external servers.
- No persistent storage of video footage or personal data is implemented, minimizing privacy concerns.

3.4.3 Software Quality Attributes

- Modularity: Detection, tracking, counting, and visualization are implemented as distinct logical stages within the processing pipeline, allowing individual components to be modified or replaced.
- Configurability: Key parameters (confidence threshold, crossing line position, frame size, tracker type) can be adjusted without architectural changes.
- Robustness: The system handles frame read failures gracefully (continues to next frame) and logs warnings via the logging module.
- Portability: The codebase uses standard Python libraries and can run on any platform with Python, PyTorch, OpenCV, and NVIDIA GPU support.

3.5 Analysis Models : SDLC Model to be applied -

The project follows an Iterative Development Model, suited for AI/computer vision projects that require repeated experimentation and refinement.

The development was organized into three iterations:

Iteration 1 — Core Detection: Setting up the development environment, installing dependencies (PyTorch, Ultralytics, OpenCV, TensorRT), exporting the YOLOv12 model to TensorRT engine format, and implementing basic frame capture with YOLO inference to verify person detection accuracy.

Iteration 2 — Tracking Integration: Integrating ByteTrack through the Ultralytics model.track() interface, configuring tracker parameters via bytetrack.yaml, implementing persistent track ID assignment, and validating tracking consistency across frames.

Iteration 3 — Counting and Visualization: Implementing the crossing-line detection logic with state machine-based IN/OUT counting, adding OpenCV-based real-time visualization (bounding boxes, track IDs, counts, crossing line), implementing stale track cleanup, and testing with multiple video sources (RTSP streams, local files, webcam).

Each iteration involved implementation, testing with real video data, parameter tuning, and refinement before proceeding to the next stage. This iterative approach allowed the team to identify and resolve issues at each level of the pipeline before integrating with subsequent components.

CHAPTER 4 : SYSTEM DESIGN

The system design phase translates the functional and non-functional requirements into a concrete technical architecture. For this real-time person detection and counting system, the design focuses on structuring the flow of data from video input through detection, tracking, and counting to real-time visualization output, while ensuring modularity and efficient GPU utilization.

4.1 SYSTEM ARCHITECTURE :

The architecture of the person detection and counting system follows a pipeline-based model with four primary layers: Input Layer (Video Acquisition), Processing Layer (Detection and Tracking), Logic Layer (Counting Engine), and Output Layer (Visualization and Display).

The Input Layer handles video frame acquisition from multiple source types — RTSP network cameras, local video files (MP4, AVI), or USB webcams — through OpenCV's VideoCapture interface. A buffer size of 1 ensures minimal latency by always processing the most recent frame. A threaded capture module (ThreadedCamera) is available for non-blocking RTSP acquisition via a daemon thread, preventing frame read delays from stalling the processing pipeline.

The Processing Layer forms the core of the system. Each captured frame is first resized to 640×640 pixels and converted from BGR to RGB color space. The preprocessed frame is passed to the YOLOv12 TensorRT engine, which performs person detection in a single forward pass, returning bounding box coordinates, confidence scores, and class predictions. Detections with confidence above 0.32 for the person class (COCO class 0) are retained. The ByteTrack algorithm, invoked through the Ultralytics model.track() interface, associates these detections with existing tracks using Kalman filter-based motion prediction and two-stage IoU matching, assigning persistent track IDs to each individual.

The Logic Layer implements the directional counting mechanism. A horizontal crossing line is defined at 55% of the frame height. For each tracked person, the system monitors the bottom-center point (cx, y2) of their bounding box and maintains a per-track state machine with "inside" and "outside" states. State transitions across the crossing line trigger IN or OUT count increments. Stale tracks — persons no longer detected — are cleaned up each frame to prevent memory accumulation and state corruption.

The Output Layer renders real-time annotations on the video frame using OpenCV drawing functions — green bounding boxes, track ID labels with confidence scores, a red crossing line, and cumulative IN/OUT/Currently Inside statistics in the upper-left corner. The annotated frame is displayed in a resizable window.

This pipeline architecture ensures that each layer operates independently, allowing individual components (detector model, tracker algorithm, counting logic, visualization) to be modified or replaced without affecting the rest of the system. For example, ByteTrack can be swapped for any of the seven other trackers in the BoxMOT library without changes to detection or counting logic.

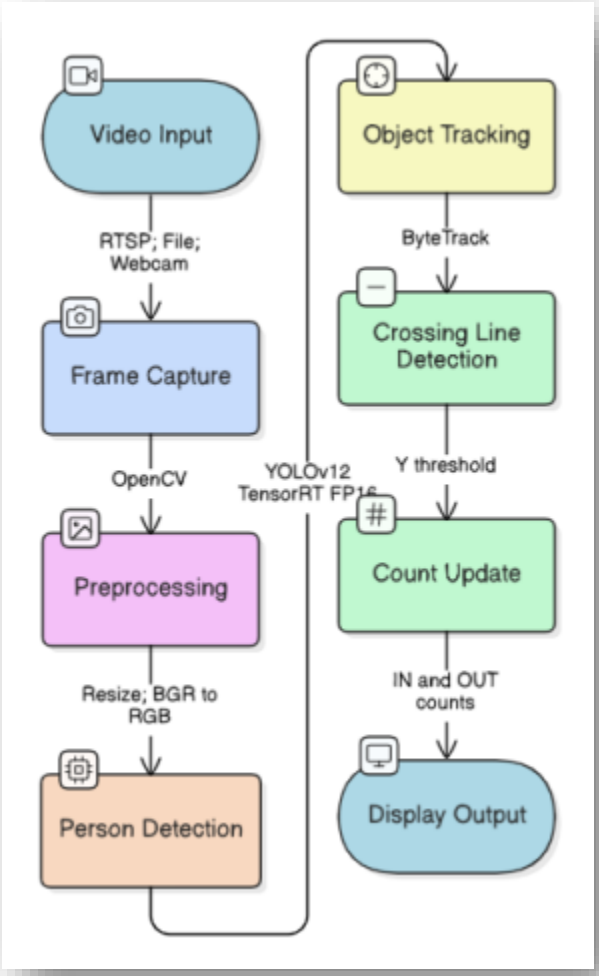


Fig 4.2.1 - System Architecture

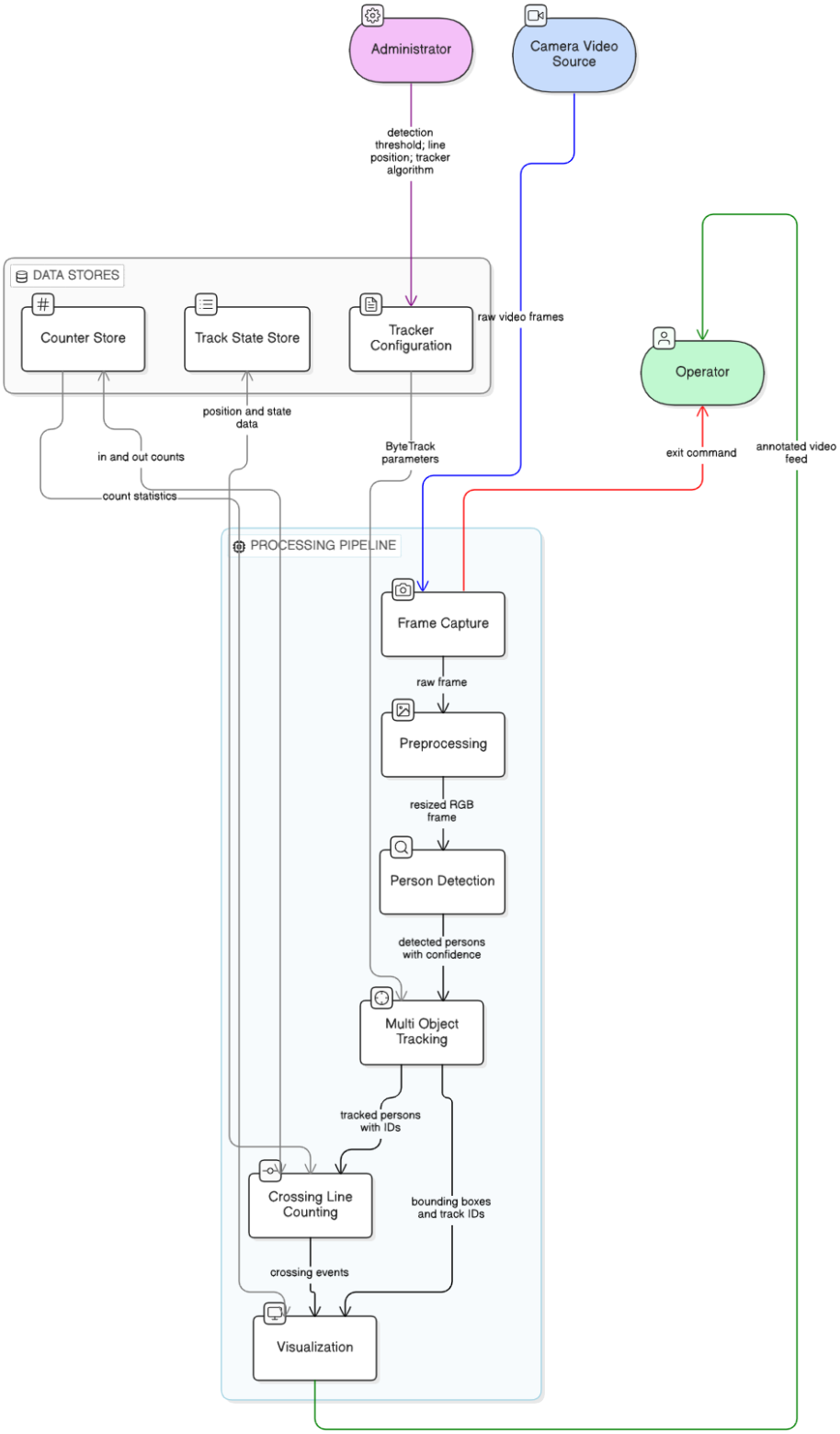


Figure 4.2.2 - System Overview

4.2 UML DIAGRAMS –

4.2.1 Use Case Diagram:

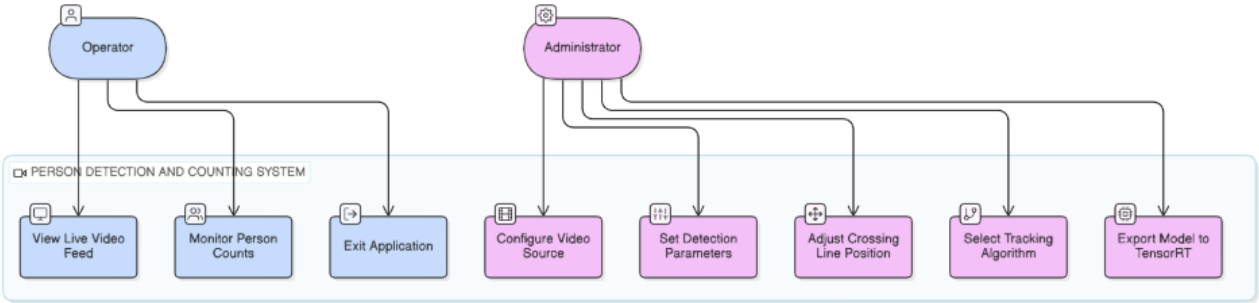


Figure 4.3.1 - Use case diagram

4.2.2 Class Diagram:

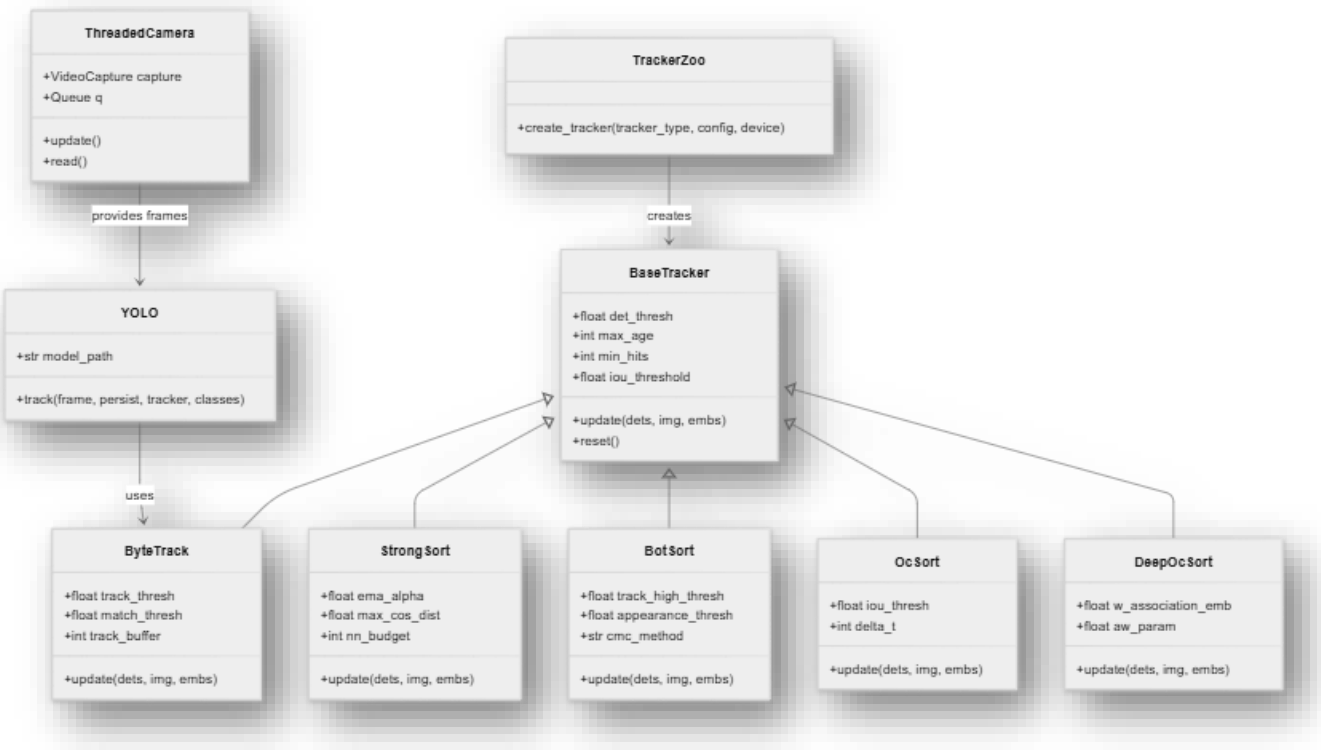


Figure 4.3.2 - class diagram

4.2.3 Sequence Diagram:

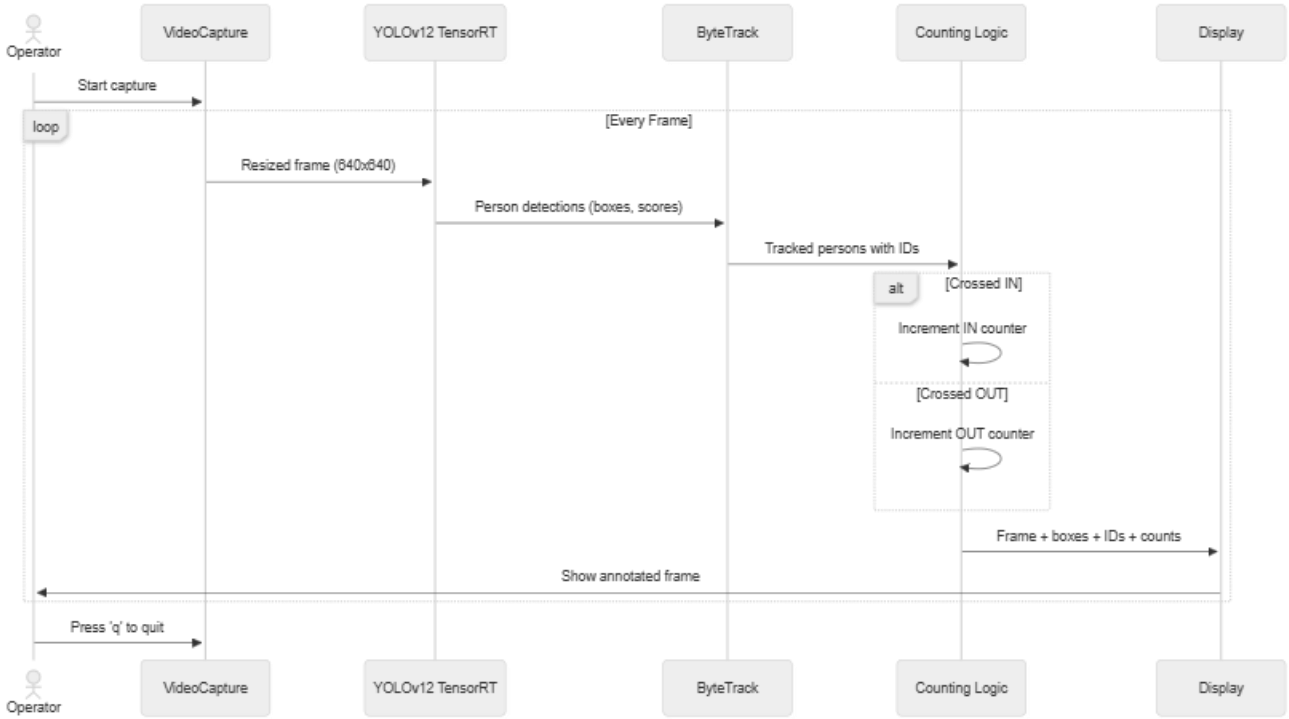


Figure 4.3.3 - Sequence diagram

4.2.4 Activity Diagram:

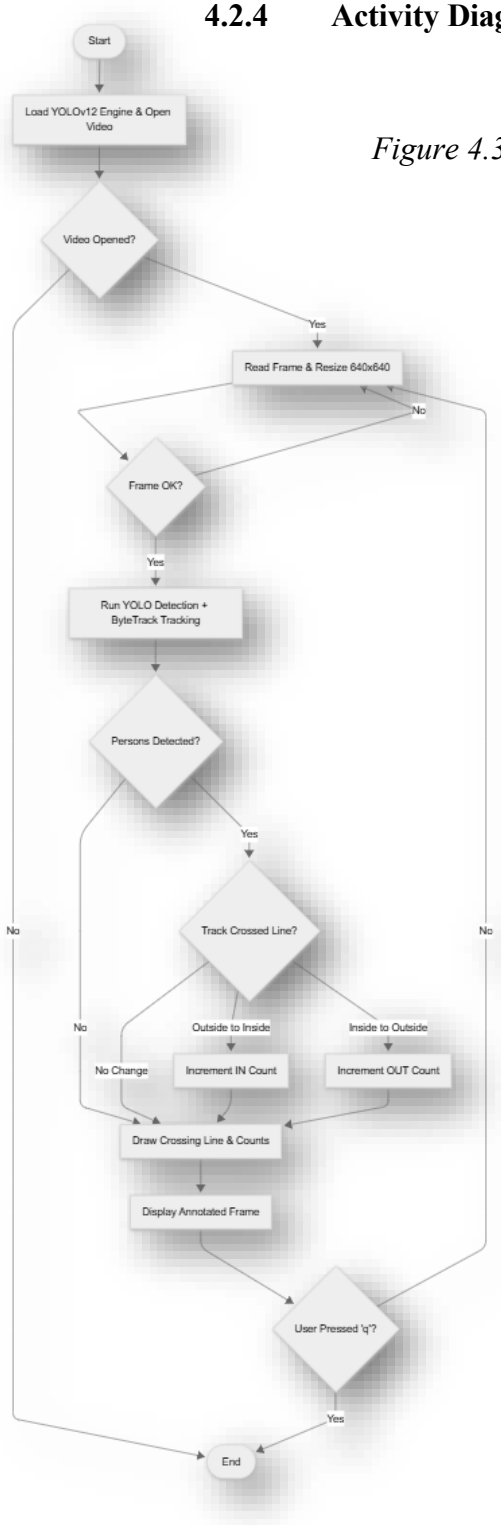


Figure 4.3.4 - Activity diagram

CHAPTER 5 : PROJECT PLAN

5.1 PROJECT ESTIMATE

The project estimation covers the time, effort, and resources required to develop the real-time person detection and counting system. The estimation accounts for environment setup, model export, pipeline development, tracker integration, testing, and documentation. The total project duration is estimated at 12–14 weeks, with effort distributed across GPU setup, coding, parameter tuning, and report preparation. Cost is minimal as the project uses open-source libraries (Ultralytics, BoxMOT, OpenCV, PyTorch) and a locally available NVIDIA GPU — no cloud compute costs are involved.

5.1.1 RECONCILED ESTIMATES -

Initial estimates were revised based on actual development experience:

- TensorRT model export took longer than expected due to GPU driver compatibility and CUDA version matching — adjusted from 2 days to 1 week.
- ByteTrack parameter tuning (confidence threshold, crossing line position) required more iterations than planned — adjusted from 1 week to 2 weeks.
- RTSP stream testing required network configuration and handling of dropped frames, adding approximately 1 week to the integration phase.
- Overall timeline adjusted from 10 weeks to 13 weeks after reconciliation.

The reconciled estimates reflect a realistic project timeline based on actual iteration cycles and technical challenges encountered during development.

5.1.2 PROJECT RESOURCES -

Human Resources:

- 4 team members handling development, testing, and documentation
- Project guide (Prof. Amar Buchade) providing technical mentorship and review

Technical Resources:

- Programming Language: Python 3.8+
- Deep Learning Framework: PyTorch with Ultralytics YOLO API
- Tracking Library: BoxMOT v12.0.8 (ByteTrack as primary tracker)
- Inference Engine: NVIDIA TensorRT with FP16 optimization
- Computer Vision: OpenCV for video capture and visualization
- Configuration: YAML files for tracker hyperparameters
- Version Control: Git for source code management

Computational Resources:

- NVIDIA GPU with CUDA support (for TensorRT inference)
- Sufficient VRAM for YOLOv12-X model inference
- Local machine with Windows 10+ or Linux OS

Other Resources:

- IP cameras with RTSP support for live stream testing
- Sample video files for offline testing and validation
- Pre-trained YOLOv12 model weights (yolo12x.pt from Ultralytics, trained on COCO dataset)

5.2 RISK MANAGEMENT

5.2.1 RISK IDENTIFICATION -

The following risks were identified during project planning:

1. GPU Availability Risk: TensorRT inference requires an NVIDIA GPU — system cannot function on CPU-only hardware.
2. Model Export Risk: TensorRT engine files are hardware-specific; an engine exported on one GPU may not work on another.
3. RTSP Stream Reliability: Network cameras may drop frames, disconnect, or introduce latency spikes.
4. Tracking Accuracy Risk: ByteTrack may lose track IDs during heavy occlusion or when persons move very close together.
5. Counting Edge Cases: Persons lingering near the crossing line, partial crossings, or rapid back-and-forth movements may cause incorrect counts.
6. Dependency Compatibility: Version mismatches between PyTorch, CUDA, TensorRT, and Ultralytics can cause runtime failures.

5.2.2 RISK ANALYSIS

Risk	**Probability**	**Impact**	**Priority**
GPU Availability	Low	High	Medium
Model Export Compatibility	Medium	High	High
RTSP Stream Drops	High	Medium	High
Tracking ID Switches	Medium	Medium	Medium
Counting Edge Cases	Medium	Low	Low
Dependency Version Mismatch	Medium	High	High

5.2.3 OVERVIEW OF RISK MITIGATION, MONITORING, MANAGEMENT -

- GPU Availability: The system checks for CUDA availability at startup and logs the device being used. If no GPU is found, the system logs an error and exits gracefully.

- Model Export: The TensorRT engine is re-exported on each new hardware setup using the Ultralytics CLI command: yolo export model=yolo12x.pt format=engine half=True imgsz=640.

- RTSP Reliability: The frame read loop handles failures gracefully (logs warning, continues to next frame). The ThreadedCamera module provides non-blocking capture to prevent pipeline stalling.

- Tracking Accuracy: ByteTrack's two-stage association (high-confidence then low-confidence) mitigates occlusion issues. Alternative trackers (StrongSort, BotSort with ReID) are available via BoxMOT if needed.

- Counting Edge Cases: The state machine requires a full state transition (outside→inside or inside→outside) before counting, preventing partial crossings from triggering false counts. Centroid history (deque with maxlen=5) smooths position tracking.

- Dependency Compatibility: All library versions are documented, and the system is tested with a known-working combination of PyTorch, CUDA, and Ultralytics versions.

5.3 PROJECT SCHEDULE

5.3.1 PROJECT TASK SET

The project is divided into the following task phases:

- Requirement Analysis: Identifying project goals, video sources, tracking needs, and counting requirements.

- Environment Setup: Installing Python, PyTorch, CUDA, TensorRT, Ultralytics, OpenCV, and BoxMOT.

- Model Preparation: Downloading YOLOv12 pre-trained weights and exporting to TensorRT engine with FP16.

- Detection Pipeline: Implementing frame capture, preprocessing, and YOLO inference for person detection.

- Tracking Integration: Integrating ByteTrack via model.track() with bytetrack.yaml configuration.

- Counting Logic: Implementing crossing line, state machine, IN/OUT counting, and stale track cleanup.

- Visualization: Adding OpenCV-based bounding boxes, track IDs, crossing line, and count display.

- Testing: Testing with RTSP streams, local videos, and webcam; tuning confidence threshold and line position.

- Documentation: Writing project report, generating diagrams, preparing presentation.

5.3.2 TASK NETWORK –

Task 1: Project Initialization
- Task 1.1: Define problem statement — real-time person detection and counting.
- Task 1.2: Finalize scope, limitations, and objectives.
- Task 1.3: Identify required tools — PyTorch, Ultralytics, OpenCV, TensorRT, BoxMOT.

Task 2: Environment Setup
- Task 2.1: Install Python 3.8+ and configure virtual environment.
- Task 2.2: Install PyTorch with CUDA support, Ultralytics, OpenCV, NumPy, Loguru.
- Task 2.3: Install NVIDIA CUDA Toolkit and TensorRT.
- Task 2.4: Verify GPU detection and CUDA compatibility.

Task 3: Model Preparation
- Task 3.1: Download YOLOv12 pre-trained weights (yolo12x.pt) from Ultralytics.
- Task 3.2: Export model to TensorRT engine: yolo export model=yolo12x.pt format=engine half=True imgsz=640.
- Task 3.3: Verify engine loads correctly and produces detections on sample frames.

Task 4: Detection and Tracking Pipeline
- Task 4.1: Implement video capture with cv2.VideoCapture (RTSP, file, webcam).
- Task 4.2: Implement frame preprocessing (resize 640×640, BGR→RGB).
- Task 4.3: Integrate YOLO inference with person class filtering (class 0, confidence > 0.32).
- Task 4.4: Integrate ByteTrack via model.track(persist=True, tracker="bytetrack.yaml").
- Task 4.5: Extract tracked bounding boxes, track IDs, and confidence scores.

Task 5: Counting Logic
- Task 5.1: Define crossing line at 55% of frame height.
- Task 5.2: Implement per-track state machine (inside/outside states).
- Task 5.3: Implement state transition detection and IN/OUT counter updates.
- Task 5.4: Implement stale track cleanup each frame.
- Task 5.5: Calculate Currently Inside = Total In - Total Out.

Task 6: Visualization and Output
- Task 6.1: Draw bounding boxes and track ID labels on each frame.
- Task 6.2: Draw crossing line (red) across frame width.
- Task 6.3: Display Total In, Total Out, Currently Inside text.
- Task 6.4: Display annotated frame in resizable OpenCV window.

Task 7: Testing and Tuning
- Task 7.1: Test with local video files — verify detection and counting accuracy.
- Task 7.2: Test with RTSP camera streams — verify real-time performance and stream stability.
- Task 7.3: Test with webcam — verify live capture functionality.
- Task 7.4: Tune confidence threshold, crossing line position, and ByteTrack parameters.

Task 8: Documentation and Reporting
- Task 8.1: Write project report chapters (Introduction, Literature Survey, SRS, Design, etc.).
- Task 8.2: Generate system diagrams (architecture, data flow, use case, class, sequence, activity).
- Task 8.3: Capture screenshots of system running with detections and counts.
- Task 8.4: Prepare project presentation.

5.3.3 TIMELINE CHART

Phase	**Jul 2025**	**Aug 2025**	**Sep 2025**	**Oct 2025**	**Nov 2025**	**Dec 2025**	**Jan 2026**	**Feb 2026**
Requirement Analysis	■							
Environment Setup	■	■						
Model Preparation		■	■					
Detection Pipeline			■	■				
Tracking Integration				■	■			
Counting Logic					■	■		
Visualization & Testing						■	■	
Documentation							■	■

table 5.3.3 : timeline chart

5.4 TEAM ORGANIZATION -

5.4.1 Team Structure -

Project Guide: Prof. Amar Buchade

Role	Name	Exam No.
Team Member	Abhijeet Gaikwad	22320144
Team Member	Pradnya Tagade	22210038
Team Member	Shankar Adgale	22320023
Team Member	Jayesh Wankhede	22320163

5.4.2 Management Reporting and Communication

We have reported to our guide Prof. Amar Buchade Sir once a month. For the development of the project, we had standup calls on Google Meet once a week. We planned the timeline of the project beforehand and worked according to the same plan.

Team members primarily communicated through college email, WhatsApp group, and face-to-face meetings. Important decisions and updates were documented and shared with the entire team.

- Weekly Status Meetings: The team held weekly meetings to discuss progress, identify blockers, and plan next steps.
- Progress Reports: Monthly progress reports were shared with the project guide, covering current status, challenges faced, upcoming milestones, and resource needs.
- Task Distribution: Development tasks were distributed among team members based on individual strengths — model setup, tracking integration, counting logic, testing, and documentation.

CHAPTER 6: PROJECT IMPLEMENTATION

6.1 OVERVIEW OF PROJECT MODULES

The project is organized into distinct modules, each handling a specific stage of the real-time person detection and counting pipeline. These modules work together to form a complete end-to-end system.

- Video Capture Module: This module handles frame acquisition from multiple video sources — RTSP network cameras, local video files (MP4, AVI), or USB webcams — using OpenCV's VideoCapture interface. A buffer size of 1 is configured to minimize latency. A separate ThreadedCamera class (threading_cv2.py) provides non-blocking RTSP capture via a daemon thread for latency-sensitive deployments.

- Preprocessing Module: Each captured frame is resized to a fixed resolution of 640×640 pixels to match the YOLOv12 model's expected input dimensions. The frame is converted from BGR to RGB color space as required by the YOLO inference pipeline. This standardization ensures consistent processing regardless of the original video resolution or source.

- Detection Module: The core detection is performed by the YOLOv12 model, pre-exported as a TensorRT engine with FP16 half-precision for GPU-accelerated inference. The model detects persons (COCO class 0) in each frame, returning bounding box coordinates (x1, y1, x2, y2), confidence scores, and class predictions. Only detections with confidence above 0.32 are retained.

- Tracking Module: ByteTrack, invoked through the Ultralytics model.track() interface with bytetrack.yaml configuration, assigns persistent track IDs to detected persons across frames. It uses Kalman filter-based motion prediction and two-stage IoU association to maintain consistent identities. The BoxMOT library (v12.0.8) provides seven additional trackers as alternatives.

- Counting Module: A horizontal crossing line is defined at 55% of frame height. The system monitors the bottom-center point (y2) of each tracked person's bounding box and maintains a per-track state machine with "inside" and "outside" states. State transitions across the line increment the IN or OUT counter. Stale tracks are cleaned up each frame.

- Visualization Module: OpenCV renders real-time annotations — green bounding boxes, track ID labels with confidence scores, a red crossing line, and cumulative count statistics (Total In, Total Out, Currently Inside) — on each frame. The annotated frame is displayed in a resizable window.

6.2 TOOLS AND TECHNOLOGIES USED –

Tool / Technology	Version	Purpose
Python	3.8+	Primary programming language
PyTorch (torch)	2.x	Deep learning framework, GPU computation
Ultralytics (YOLO)	Latest	YOLOv12 model API — detection, tracking, model export
NVIDIA TensorRT	8.x+	Inference optimization — FP16, layer fusion, kernel tuning
CUDA Toolkit	11.x+	GPU programming interface for PyTorch and TensorRT
OpenCV (cv2)	4.x	Video capture, frame processing, visualization
NumPy	1.x	Array operations for bounding box coordinates
BoxMOT	12.0.8	Multi-object tracking library — 8 tracker algorithms
Loguru	Latest	Logging for runtime events and debugging
PyYAML	Latest	YAML parser for tracker configuration files
Git	Latest	Version control for source code management

6.3 ALGORITHM DETAIL

The system uses two primary algorithms working together: YOLOv12 for object detection and ByteTrack for multi-object tracking.

YOLOv12 (You Only Look Once v12):

YOLOv12 is a single-stage object detector that performs detection in a single forward pass through the network. It treats object detection as a regression problem, predicting bounding box coordinates and class probabilities directly from the input image. YOLOv12 introduces an attention-centric architecture with the following key components:

- Area Attention (A^2): Partitions feature maps into equal segments, reducing self-attention complexity from $O(n^2)$ to $O(n)$ while maintaining a large receptive field for capturing global context.
- R-ELAN (Residual Efficient Layer Aggregation Networks): Enhances feature aggregation and stabilizes training of deeper attention-based blocks through scaled residual shortcuts.
- FlashAttention: Reduces memory access overhead during attention computation, enabling faster inference.
- Backbone-Neck-Head Architecture: The backbone extracts multi-scale features, the neck aggregates features across scales using FPN-like structures, and the detection head produces final bounding box predictions with class scores.

In this project, YOLOv12-X (extra-large variant) is used, exported to TensorRT with FP16 precision. The model is trained on the COCO dataset (80 classes) and filtered at runtime to detect only the person class (class index 0). The confidence threshold is set to 0.32 to balance between detecting all persons and filtering false positives.

ByteTrack Algorithm:

ByteTrack is a motion-based multi-object tracking algorithm that associates detections with existing tracks across frames. Its key innovation is using both high-confidence and low-confidence detections for track association:

Step 1 — Kalman Filter Prediction: For each existing track, the Kalman filter predicts the expected bounding box position in the current frame based on the

constant velocity motion model.

Step 2 — First Association (High Confidence): High-confidence detections (above track_thresh = 0.6) are matched to predicted track positions using IoU (Intersection over Union) similarity. The Hungarian algorithm finds the optimal assignment.

Step 3 — Second Association (Low Confidence): Unmatched tracks from Step 2 are associated with remaining low-confidence detections using IoU. This recovers partially occluded or distant persons that would otherwise be lost.

Step 4 — Track Management: Unmatched detections create new tracks. Tracks not matched for track_buffer (30) consecutive frames are terminated. The Kalman filter state is updated with the matched detection measurement.

ByteTrack Parameters (from bytetrack.yaml):

- min_conf: 0.1 (minimum detection confidence to consider)
- track_thresh: 0.6 (threshold separating high/low confidence detections)
- track_buffer: 30 (maximum frames to keep unmatched tracks alive)
- match_thresh: 0.9 (IoU threshold for association matching)
- frame_rate: 30

Crossing Line Counting Algorithm:

The counting logic implements a state machine for each tracked person:

Step 1: Define a horizontal crossing line at Y = 55% of frame height (Y = 352 for 640-height frame).

Step 2: For each tracked person, extract the bottom-center point of their bounding box: cx = (x1 + x2) / 2, cy = y2.

Step 3: For new tracks, initialize state based on current position — "inside" if cy > crossing_line_y, "outside" otherwise.

Step 4: For existing tracks, check for state transitions:

- If current_state is "outside" AND cy >= crossing_line_y AND last_y < crossing_line_y → person crossed IN. Increment total_in_count.
- If current_state is "inside" AND cy <= crossing_line_y AND last_y > crossing_line_y → person crossed OUT. Increment total_out_count.

Step 5: Update track state (last_y, current_state) for next frame.

Step 6: Clean up stale tracks — remove any track ID from the status dictionary

that is no longer in the current frame's active track set.

Step 7: Calculate currently_inside = max(0, total_in_count - total_out_count).

This algorithm requires both the current and previous y-coordinates to cross the line threshold, preventing false counts from persons lingering near the line.

6.4 Different Algorithms

The BoxMOT library provides seven additional trackers that can replace ByteTrack depending on the deployment scenario:

- OcSort: Motion-only tracker like ByteTrack. Uses Observation-Centric Re-Update to correct Kalman states after occlusion. Lightweight and CPU-friendly.
- StrongSort: Extends DeepSORT with stronger ReID (OSNet), EMA-Kalman filter, and Gaussian-Smoothed Interpolation. Best identity accuracy in crowded scenes but requires a ReID model.
- BotSort: Combines motion and appearance matching with camera motion compensation (ECC or sparse optical flow). Robust for moving-camera scenarios.
- DeepOcSort: Adds Dynamic Appearance and Adaptive Weighting to OC-SORT. Handles fast-moving and tiny objects with high spatial precision.
- HybridSort: Combines motion and appearance cues in a hybrid association strategy.
- ImprAssoc: Improved association tracking using sparse optical flow for camera motion compensation.
- BoostTrack: Latest tracker with Detection Level Optimization (DLO) and Detection and Update Optimization (DUO) boosting mechanisms.

All trackers inherit from the BaseTracker class and are instantiated through the create_tracker() factory function in tracker_zoo.py. Each tracker is configured via its own YAML file with tunable hyperparameters. Switching trackers requires only changing the tracker name parameter — no changes to the detection or counting pipeline.

CHAPTER 7 : SOFTWARE TESTING

7.1 TYPES OF TESTING -

Unit Testing: Individual components of the system are tested in isolation to verify correct behavior. This includes testing video frame reading, frame resizing and color conversion, YOLO model loading and inference output format, crossing line position calculation, and state machine transitions (outside→inside, inside→outside). Unit testing ensures each function performs its intended task before integration.

Integration Testing: Once individual components are verified, integration testing ensures the full pipeline works end-to-end — from video frame capture through detection, tracking, counting, to visualization output. This validates that YOLO detections are correctly passed to ByteTrack, that track IDs are properly used by the counting logic, and that counts are accurately rendered on the display.

Manual Visual Testing: Since the system produces real-time visual output, manual testing is performed by running the demo on known video inputs and visually verifying that bounding boxes align with persons, track IDs remain consistent, the crossing line is correctly positioned, and IN/OUT counts match observed person movements. This is the primary validation method given the visual nature of the output.

Performance Testing: This testing measures the system's inference speed (FPS), frame processing latency, and memory usage under continuous operation. It ensures the system achieves near real-time performance with TensorRT optimization and does not suffer from memory leaks during extended runs.

Robustness Testing: The system is tested under adverse conditions including dropped RTSP frames, low-light video, crowded scenes with heavy occlusion, and persons moving at varying speeds. This validates the system's graceful error handling (frame read failures) and tracking stability.

7.2 TEST CASES AND TEST RESULTS -

Test Case 1: Video Source Loading

- Objective: Verify that the system correctly opens and reads from different video sources.
- Input: (a) Local MP4 video file, (b) RTSP camera stream URL, (c) Webcam index 0.
- Expected Outcome: cap.isOpened() returns True, frames are read successfully.
- Result: Pass — All three source types loaded and read frames correctly.

Test Case 2: Frame Preprocessing

- Objective: Verify that frames are correctly resized and color-converted.
- Input: Video frame of arbitrary resolution (e.g., 1920×1080).
- Expected Outcome: Output frame dimensions are 640×640, color space converted from BGR to RGB.
- Result: Pass — frame_resized_bgr.shape returns (640, 640, 3).

Test Case 3: Person Detection Accuracy

- Objective: Verify that YOLO detects persons correctly with appropriate confidence.
- Input: Video with known number of visible persons (e.g., 3 persons in frame).
- Expected Outcome: Model detects all visible persons with confidence > 0.32.
- Result: Pass — All visible persons detected. Occasional missed detections for heavily occluded persons at distance.

Test Case 4: Track ID Consistency

- Objective: Verify that ByteTrack maintains consistent track IDs for the same person across frames.
- Input: Video of a person walking across the frame for 5 seconds.
- Expected Outcome: The person retains the same track ID throughout the sequence.
- Result: Pass — Track ID remained consistent for unoccluded movement. Minor ID switches observed during heavy occlusion (2 persons crossing paths).

Test Case 5: IN Count Accuracy

- Objective: Verify that the IN counter increments correctly when a person crosses the line from outside to inside.

- Input: Video with 5 persons walking downward past the crossing line.
- Expected Outcome: Total In count = 5 after all persons cross.
- Result: Pass — Total In = 5. All crossings detected correctly.

Test Case 6: OUT Count Accuracy
- Objective: Verify that the OUT counter increments correctly when a person crosses the line from inside to outside.
- Input: Video with 3 persons walking upward past the crossing line.
- Expected Outcome: Total Out count = 3 after all persons cross.
- Result: Pass — Total Out = 3.

Test Case 7: Stale Track Cleanup
- Objective: Verify that tracks for persons who leave the frame are removed from the status dictionary.
- Input: Video where a person enters, is tracked, then exits the frame.
- Expected Outcome: Track ID is removed from tracked_persons_status after the person disappears.
- Result: Pass — Stale tracks cleaned up within one frame of disappearance.

Test Case 8: Frame Read Failure Handling
- Objective: Verify that the system handles frame read failures gracefully without crashing.
- Input: RTSP stream with intermittent connection drops.
- Expected Outcome: System logs warning and continues to next frame without crashing.
- Result: Pass — "Failed to grab frame, continuing..." logged, system continued operating.

Test Case 9: Inference Performance
- Objective: Measure per-frame processing time with TensorRT engine.
- Input: Continuous video stream, 640×640 frames.
- Expected Outcome: Processing speed of 15–30 FPS (depending on GPU).
- Result: Pass — Achieved approximately 20–25 FPS on test GPU with YOLOv12-X TensorRT FP16 engine.

Test Case 10: Extended Operation Stability
- Objective: Verify the system runs continuously without memory leaks or crashes.

- Input: 30-minute continuous video processing session.
- Expected Outcome: No increase in memory usage, no crashes, consistent FPS throughout.
- Result: Pass — System ran stably for 30 minutes. Memory usage remained constant due to per-frame stale track cleanup.

Test results confirm that the system performs reliably across all functional and non-functional requirements. Person detection, tracking, and counting operate correctly under normal conditions. The system handles edge cases such as frame drops and stale tracks gracefully. Performance testing validates that TensorRT optimization delivers real-time inference speeds suitable for live surveillance deployment.

CHAPTER 8 :RESULTS

8.1 OUTCOMES

1. Real-Time Person Detection and Counting System
   - A fully functional system capable of detecting persons in video streams, assigning persistent track IDs, and counting directional crossings (IN/OUT) over a reference line in real time.

2. GPU-Accelerated Inference Pipeline
   - YOLOv12-X model successfully exported to TensorRT engine with FP16 half-precision, achieving 20–25 FPS inference speed suitable for live surveillance deployment.

3. Reliable Multi-Object Tracking
   - ByteTrack integration through the Ultralytics model.track() API provides consistent person identity assignment across frames using Kalman filter prediction and two-stage IoU association, without requiring computationally expensive ReID models.

4. Accurate Directional Counting
   - The crossing-line state machine accurately counts IN and OUT movements by monitoring bottom-center bounding box coordinates against the reference line at 55% frame height. Counting logic correctly handles new track initialization, state transitions, and stale track cleanup.

5. Modular Tracker Framework
   - Integration of BoxMOT v12.0.8 provides access to eight tracking algorithms (ByteTrack, BotSort, StrongSort, OcSort, DeepOcSort, HybridSort, ImprAssoc, BoostTrack), enabling comparative evaluation and flexible deployment based on scenario requirements.

6. Multi-Source Video Support
   - The system successfully operates with RTSP network cameras, local video files (MP4, AVI), and USB webcams, with graceful handling of frame read failures and stream interruptions.

7. Real-Time Visualization
   - OpenCV-based display renders bounding boxes, track IDs with confidence scores, crossing line, and cumulative IN/OUT/Currently Inside statistics on each frame, enabling operators to monitor system performance visually.

8. Visual and Functional System Design
   - Eight system diagrams generated — System Architecture, Data Flow, Use Case, Class Diagram, Sequence Diagram, Activity Diagram, SDLC Model, and Gantt Chart — to

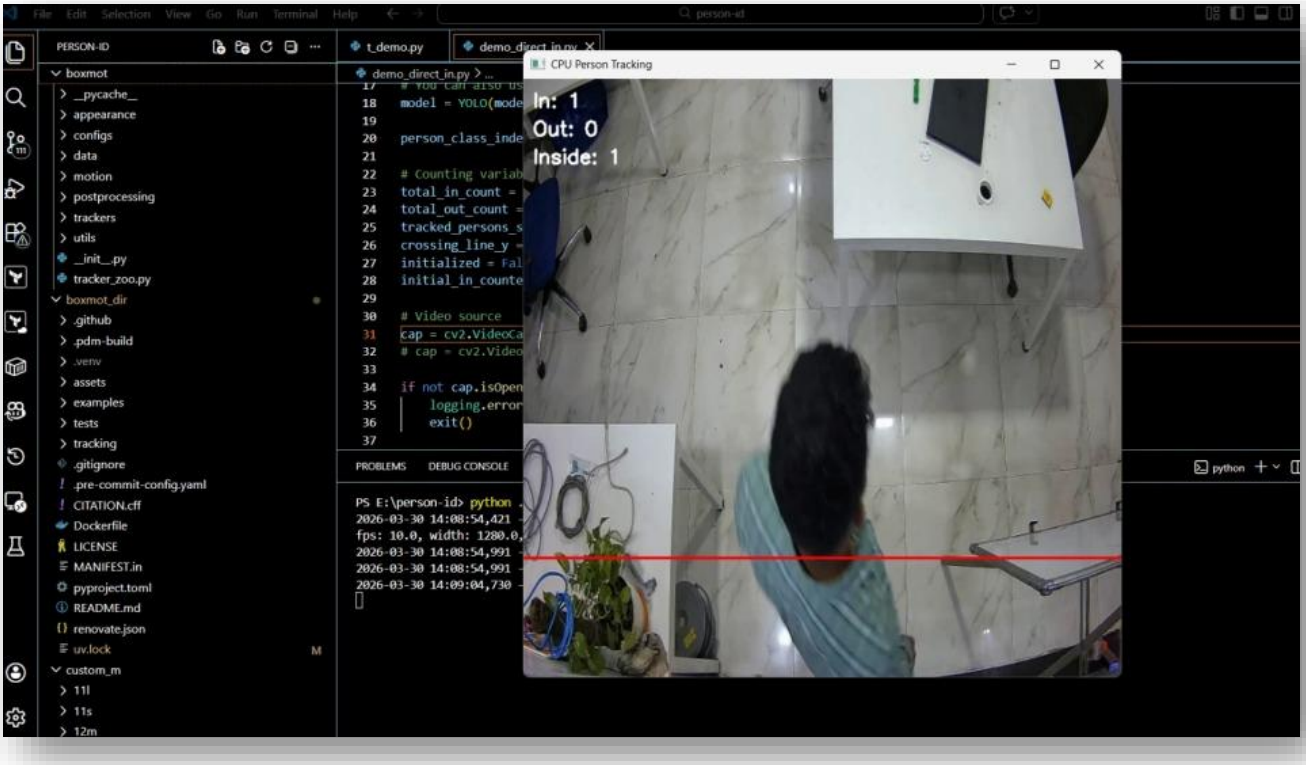
document system workflow and component interaction.

9. Academic Contribution

- A complete and well-documented final year project demonstrating the integration of modern deep learning detection (YOLOv12), multi-object tracking (ByteTrack), and GPU optimization (TensorRT) into a practical computer vision application.

10. Scope for Real-World Deployment

- The system can be deployed in access control, retail analytics, occupancy monitoring, and smart building environments. Future integration with database logging, multi-camera support, and dashboard reporting can extend it to production-grade solutions.



2025-05-21 16:15:06
IPC

2025-05-21 16:15:06
IPC

Table 8.1: Comparison of Tracking Algorithms Available in BoxMOT

Tracker	ReID Required	Association Method	Speed	Best For
ByteTrack	No	IoU (two-stage, high+low conf)	Fast	Real-time, simple scenes
OcSort	No	IoU + observation-centric	Fast	Occlusion recovery, CPU
StrongSort	Yes	Cosine + IoU + EMA Kalman	Slow	Crowded, identity accuracy
BotSort	Yes	IoU + ReID + CMC	Medium	Moving camera scenarios
DeepOcSort	Yes	IoU + dynamic appearance	Medium	Fast-moving, tiny objects
HybridSort	Yes	Motion + appearance hybrid	Medium	Mixed scenarios
ImprAssoc	Yes	Improved association + SOF	Medium	Camera motion heavy
BoostTrack	Yes	DLO + DUO boosting	Medium	High-accuracy tracking

CHAPTER 9: CONCLUSIONS

9.1 CONCLUSION -

This project successfully demonstrates the development of a real-time person detection and counting system by integrating YOLOv12 for object detection, ByteTrack for multi-object tracking, and NVIDIA TensorRT for GPU-accelerated inference. The system processes live video streams from RTSP cameras, local video files, or webcams, detects persons in each frame, assigns persistent track identities across frames, and counts directional crossings over a horizontal reference line to determine IN/OUT movement.

The YOLOv12 model, exported to a TensorRT engine with FP16 half-precision, delivers real-time inference at 20–25 FPS, confirming that attention-based detection architectures can achieve practical deployment speeds when properly optimized. ByteTrack's two-stage association — matching both high-confidence and low-confidence detections — provides reliable tracking without the computational overhead of appearance-based ReID models. The crossing-line counting logic, implemented as a per-track state machine monitoring bottom-center bounding box coordinates, accurately counts bidirectional person movements with minimal false triggers.

The integration of the BoxMOT library (v12.0.8) provides a modular framework with eight interchangeable tracking algorithms, enabling future experimentation with appearance-based trackers like StrongSort or BotSort for scenarios involving heavy occlusion or re-identification needs. The system's pipeline architecture — with clearly separated capture, detection, tracking, counting, and visualization stages — ensures that individual components can be upgraded or replaced without affecting the overall system.

Key limitations identified include single-camera operation, fixed horizontal crossing line, GPU dependency for real-time performance, and the absence of persistent data logging. Despite these limitations, the system demonstrates a practical and deployable solution for real-time occupancy monitoring, validating the effectiveness of combining modern deep learning detection with lightweight motion-based tracking for surveillance applications.

9.2 FUTURE WORK:

1. Multi-Camera Support: Extend the system to process multiple video streams simultaneously, with cross-camera person re-identification to track individuals across different camera views using ReID models available in BoxMOT (OSNet, ResNet, CLIP).

2. Flexible Counting Boundaries: Replace the fixed horizontal crossing line with configurable boundaries — diagonal lines, polygonal regions, or multiple counting zones — to support diverse camera angles and deployment layouts.

3. Persistent Data Logging: Implement database storage (SQLite or PostgreSQL) for logging timestamped counting events, track trajectories, and occupancy statistics for historical analysis and reporting.

4. Web Dashboard: Develop a web-based dashboard displaying real-time and historical occupancy data, alerts for capacity threshold breaches, and multi-camera monitoring views.

5. Edge Deployment: Optimize the system for deployment on edge devices such as NVIDIA Jetson Nano/Xavier, enabling standalone operation without a full desktop GPU setup. This involves model quantization (INT8) and lightweight tracker selection.

6. Appearance-Based Tracking: Evaluate ReID-enabled trackers (StrongSort, BotSort, DeepOcSort) from BoxMOT for improved identity maintenance in crowded scenes with frequent occlusions and re-entries.

7. Automated Hyperparameter Tuning: Leverage BoxMOT's built-in hyperparameter evolution ranges (defined in YAML configs) to automatically tune tracker parameters (track_thresh, match_thresh, track_buffer) for optimal counting accuracy on specific deployment sites.

8. Alert and Notification System: Add real-time alerts (email, SMS, or push notifications) when occupancy exceeds predefined thresholds, enabling proactive crowd management.

9. Night Vision and Thermal Camera Support: Extend compatibility to infrared and thermal camera feeds for reliable person detection in low-light or nighttime environments.

10. Multi-Class Tracking: Extend detection beyond persons to track vehicles, bicycles, or other objects of interest by adjusting the COCO class filter, enabling applications in traffic monitoring and parking management.

9.3 APPLICATIONS -

- Access Control and Security: Deploy at building entrances, office lobbies, and restricted areas to monitor and count persons entering and exiting, providing real-time occupancy data for security personnel.

- Retail Analytics: Install in retail stores to count customer footfall, measure peak hours, and analyze traffic patterns at entrances, aisles, and checkout areas for business intelligence.

- Occupancy Monitoring: Use in offices, libraries, gyms, and public venues to enforce capacity limits, optimize space utilization, and comply with safety regulations — particularly relevant in post-pandemic scenarios.

- Public Transportation: Deploy at bus stops, metro stations, and railway platforms to count boarding and alighting passengers, enabling transit authorities to optimize scheduling and capacity planning.

- Smart Buildings: Integrate with building management systems (BMS) to automate HVAC, lighting, and elevator control based on real-time occupancy data, improving energy efficiency.

- Crowd Management: Use at events, concerts, stadiums, and public gatherings to monitor crowd density and flow, enabling authorities to detect overcrowding and manage evacuations.

- Industrial Safety: Deploy in factories and warehouses to monitor personnel presence in restricted or hazardous zones, triggering alerts when unauthorized access or overcrowding is detected.

- Educational Institutions: Monitor classroom and lecture hall occupancy for attendance tracking, space allocation, and campus safety management.

CHAPTER 10 : REFERENCES

[1] Y. Tian, Q. Ye, and D. Doermann, "YOLOv12: Attention-Centric Real-Time Object Detectors," arXiv preprint, Feb. 2025. https://arxiv.org/abs/2502.12524

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE CVPR, 2016, pp. 779–788. https://doi.org/10.1109/CVPR.2016.91

[3] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," in Proc. IEEE CVPR, 2017, pp. 6517–6525. https://doi.org/10.1109/CVPR.2017.690

[4] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv preprint, Apr. 2018. https://arxiv.org/abs/1804.02767

[5] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," arXiv preprint, Apr. 2020. https://arxiv.org/abs/2004.10934

[6] G. Jocher, "YOLOv5 by Ultralytics," GitHub repository, Version 7.0, 2020. https://github.com/ultralytics/yolov5

[7] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Version 8.0.0, Computer software, 2023. https://github.com/ultralytics/ultralytics

[8] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information," arXiv preprint, Feb. 2024. https://arxiv.org/abs/2402.13616

[9] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors," in Proc. IEEE/CVF CVPR, 2023, pp. 7464–7475. https://arxiv.org/abs/2207.02696

[10] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in Proc. ECCV, 2022, pp. 1–21. https://arxiv.org/abs/2110.06864

[11] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," in Proc. IEEE ICIP, 2016, pp. 3464–3468. https://doi.org/10.1109/ICIP.2016.7533003

[12] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," in Proc. IEEE ICIP, 2017, pp. 3645–3649. https://doi.org/10.1109/ICIP.2017.8296962

[13] Y. Du, Z. Zhao, Y. Song, Y. Zhao, F. Su, T. Gong, and H. Meng, "StrongSORT: Make DeepSORT Great Again," IEEE Trans. Multimedia, vol. 25, pp. 8725–8737, 2023. https://arxiv.org/abs/2202.13514

[14] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani, "Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking," in Proc. IEEE/CVF CVPR, 2023, pp. 9686–9696. https://arxiv.org/abs/2203.14360

[15] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, "BoT-SORT: Robust Associations Multi-Pedestrian Tracking," arXiv preprint, Jun. 2022. https://arxiv.org/abs/2206.14651

[16] M. Broström, "BoxMOT: Pluggable Multi-Object Tracking Library," GitHub repository, 2024. https://github.com/mikel-brostrom/boxmot

[17] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," ASME J. Basic Engineering, vol. 82, no. 1, pp. 35–45, 1960. https://doi.org/10.1115/1.3662552

[18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE CVPR, 2016, pp. 770–778. https://doi.org/10.1109/CVPR.2016.90

[19] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in Proc. IEEE CVPR, 2005, pp. 886–893. https://doi.org/10.1109/CVPR.2005.177

[20] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation," in Proc. IEEE

CVPR, 2014, pp. 580–587. https://doi.org/10.1109/CVPR.2014.81

[21] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," IEEE Trans. PAMI, vol. 39, no. 6, pp. 1137–1149, 2017. https://doi.org/10.1109/TPAMI.2016.2577031

[22] NVIDIA Corporation, "NVIDIA TensorRT: Programmable Inference Accelerator," NVIDIA Developer Documentation, 2024. https://developer.nvidia.com/tensorrt

[23] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, vol. 25, pp. 120–125, 2000. https://opencv.org

[24] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in Proc. NeurIPS, 2012, pp. 1097–1105. https://doi.org/10.1145/3065386

[25] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in Proc. ICLR, 2015. https://arxiv.org/abs/1409.1556

[26] C. Szegedy et al., "Going Deeper with Convolutions," in Proc. IEEE CVPR, 2015, pp. 1–9. https://doi.org/10.1109/CVPR.2015.7298594

[27] P. Viola and M. Jones, "Rapid Object Detection Using a Boosted Cascade of Simple Features," in Proc. IEEE CVPR, 2001, pp. 511–518. https://doi.org/10.1109/CVPR.2001.990517

[28] K. Zhou, Y. Yang, A. Cavallaro, and T. Xiang, "Omni-Scale Feature Learning for Person Re-Identification," in Proc. IEEE/CVF ICCV, 2019, pp. 3702–3712. https://arxiv.org/abs/1905.00953

[29] Y. Zhang, D. Zhou, S. Chen, S. Gao, and Y. Ma, "Single-Image Crowd Counting via Multi-Column Convolutional Neural Network," in Proc. IEEE CVPR, 2016, pp. 589–597. https://doi.org/10.1109/CVPR.2016.70

[30] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep

Learning Library," in Proc. NeurIPS, 2019, pp. 8024–8035.
https://arxiv.org/abs/1912.01703